

PORTADA

Calidad de Software

Semana 1 — Qué es calidad y qué es un defecto

Universidad de Antioquia · Ingeniería de Sistemas



ENCUADRE

Cómo funciona este curso

El bloque semanal son 4 horas divididas en dos

- 2 horas de cátedra — con el docente, en vivo
- 2 horas de taller — en salas de Zoom asignadas, en grupo
- El entregable se sube **dentro** de esas dos horas. No después.
- Antes de cada cátedra hay material que leer. La clase asume que ya lo leyeron.

The Zoom logo, consisting of the word "zoom" in a lowercase, rounded, sans-serif font, colored in a vibrant blue.

ENCUADRE

Cómo se califica

Rubro	Peso
Seguimiento semanal (15 entregas, se descartan las 2 más bajas)	20 %
Trabajo 1 — Plan de pruebas y diseño de casos	25 %
Trabajo 2 — Ejecución, métricas, CI y automatización	25 %
Fábrica-Escuela (3 entregables)	30 %

Al entregar, cada integrante reporta en privado quién trabajó.

Quien no asista a la sala y no participe en la entrega obtiene 0 esa semana.



APERTURA

45 minutos
440 millones de dólares



Knight Capital, en cifras

Empresa	Knight Capital Americas LLC — bróker-dealer, Jersey City, NJ
Peso en el mercado	~10 % de todo el trading de acciones listadas en EE. UU.
Sistema implicado	SMARS — router algorítmico de órdenes de alta velocidad
Fecha	1 de agosto de 2012, 9:30–10:15 a. m.
Duración	~45 minutos
Pérdida (SEC)	Más de 460 millones de dólares
Sanción SEC	12 millones de dólares, censura y orden de cese
Desenlace	2013: Knight se fusiona con GETCO — deja de existir como empresa independiente

La cifra de **440 millones** que abrió esta sesión es la que Knight Capital comunicó antes de impuestos. La orden de la SEC —la fuente oficial— dice "**más de 460 millones**".

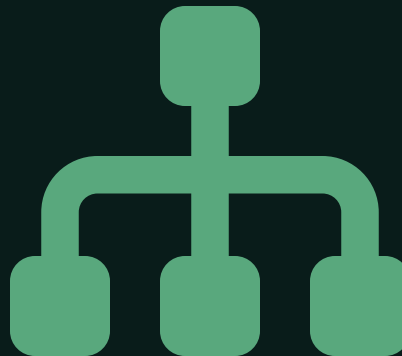
CASO · CONTEXTO TÉCNICO

Cómo funciona SMARS

SMARS recibe **órdenes padre** desde otros componentes de la plataforma y las descompone en **órdenes hija** que envía al mercado hasta completar la orden padre.

El mecanismo de parada es una **función de cantidad acumulada**: cuenta cuántas acciones de la orden padre ya se ejecutaron y detiene el envío de hijas al llegar al total.

Ese contador es la pieza que falla en este caso — y no porque tuviera un error de lógica.



CASO · ANATOMÍA TÉCNICA

Los cuatro ingredientes del desastre

1. **2003 — Código muerto que sigue vivo.** Knight deja de usar *Power Peg*, pero no borra el código de producción: sigue ahí, invocable.
2. **2005 — Un cambio sin regresión.** La función de cantidad acumulada se mueve de lugar. Nadie vuelve a probar *Power Peg*. Queda sin condición de parada: un defecto latente, invisible durante siete años.
3. **Julio 2012 — Una bandera reutilizada.** Para el nuevo programa RLP, Knight reutiliza la misma bandera que antes activaba *Power Peg*.
4. **27 jul – 1 ago 2012 — Un despliegue manual incompleto.** El código nuevo se copia a los servidores en días distintos. Un técnico omite uno de los ocho. No existe un procedimiento escrito que exija revisión por una segunda persona.

Ningún ingrediente por sí solo hunde a la empresa. Juntos, sí.

CASO · CRONOLOGÍA

La cadena causal del 1 de agosto

1. Llegan órdenes con la bandera activada.
2. Los siete servidores actualizados las procesan con normalidad.
3. El octavo interpreta la bandera como "activar *Power Peg*" e invoca el código de 2003.
4. Como el contador se movió en 2005, *Power Peg* no sabe cuándo detenerse: dispara órdenes hija sin parar.
5. Otra parte del sistema sí sabía que las órdenes ya estaban completas — esa información nunca llegó a SMARS.



CASO · PUNTO CIEGO

Las 97 alertas que nadie leyó

Desde las **8:01 a. m.** — 89 minutos antes de la apertura — un sistema interno envía correos automáticos con el mensaje "Power Peg disabled". Se enviaron **97** antes de las 9:30.

"Knight no diseñó esos mensajes como alertas de sistema, y el personal de Knight, por lo general, no los revisaba al recibirlos." — Orden de la SEC

Los mensajes existían, llegaban en tiempo real, señalaban la causa exacta — y no eran alertas.



CASO · PUNTO DE GIRO

La respuesta que empeoró el incidente

Knight no tenía procedimientos de respuesta a incidentes. Con el sistema ya enviando órdenes erróneas, el equipo intentó diagnosticar en producción — y en uno de esos intentos **desinstaló el código nuevo de los siete servidores donde sí funcionaba bien.**

Resultado: las órdenes entrantes empezaron a activar Power Peg también en esos siete servidores. Un rollback sin diagnóstico multiplicó el daño por ocho.

"Knight necesitaba una guía clara sobre cuándo desconectar del mercado un sistema que está fallando." — Orden de la SEC



CASO

Controles que existían — y por qué no sirvieron

Control existente	Por qué no funcionó
Anti-órdenes erróneas en cliente, OMS y ejecución interna	Estaban antes de SMARS; ninguno después, justo antes de salir al mercado
Tope de precio (9,5 % del mejor precio nacional)	No aplicaba a órdenes recibidas antes de la apertura — exactamente las 212 que fallaron
Límite de posición de 2 millones USD	No estaba enlazado a ningún control automático: era un número sin consecuencia
Monitoreo de posiciones (PMON)	Sin alertas automáticas, y se degradaba justo en los momentos de mayor volumen

Knight sí tenía controles. El problema no fue su ausencia: fue su ubicación, su acoplamiento y su comportamiento bajo carga.

TRANSICIÓN

Tres cosas distintas pasaron aquí

Qué ocurrió	Dónde estuvo
Omitir el octavo servidor. Reutilizar un bit ya usado.	En la cabeza de una persona
El octavo servidor con <i>Power Peg</i> activable	En el sistema desplegado
4 millones de operaciones erróneas en 45 minutos	En el comportamiento observado

Estas tres cosas tienen nombres distintos. Confundirlos es el error más común del curso.



CONCEPTO

Error, defecto, falla

Error (*error, mistake*)

Acción humana que produce un resultado incorrecto. Ocurre en la persona.

Defecto (*defect, fault, bug*)

La huella que ese error deja en un artefacto: código, documento, configuración.

Falla (*failure*)

La desviación observable del comportamiento cuando el defecto se ejecuta.



CONCEPTO

Aplicado al caso

Error	El técnico omite el octavo servidor. Alguien decide reutilizar un bit asociado a código muerto.
Defecto	El octavo servidor conserva <i>Power Peg</i> activable por la bandera reutilizada.
Falla	4 millones de operaciones erróneas durante 45 minutos.



CONTRA EJEMPLO

Un defecto que no falla

```
def calcular_descuento(monto, tipo_cliente):  
    if tipo_cliente == "premium":  
        return monto * 0.20  
    elif tipo_cliente == "corporativo":  
        return monto * 1.30      # defecto: recarga en vez de descuento  
    return monto * 0.05
```

Si ningún cliente corporativo compra, el sistema nunca falla.

El defecto está ahí desde el primer día.



CONTRA EJEMPLO

El caso inverso

¿Puede haber una falla sin defecto en el código?

- El código cumple la especificación al pie de la letra
- La especificación estaba equivocada
- El usuario observa un comportamiento incorrecto

Hay falla. El defecto está en el documento de requisitos, no en el código.



INTERACCIÓN

Ejercicio de clasificación

Recordatorio rápido — ¿dónde ocurrió cada cosa?

- **Error** — en la persona, al equivocarse
- **Defecto** — en el artefacto: código, documento, configuración
- **Falla** — en el comportamiento, cuando el defecto se ejecuta

Clasifica cada situación como error, defecto o falla:

1. El analista escribe "el campo acepta hasta 10 caracteres" cuando el negocio pide 20.
2. El formulario corta el nombre del usuario en el carácter 10.
3. Un usuario reporta que su apellido aparece incompleto en la factura.
4. El código valida `edad > 18` cuando la regla dice `edad >= 18`.

5 minutos · respondan en el chat o en voz alta

INTERACCIÓN

Cambio de tema

¿Esta aplicación es de calidad?

(proyectar la pantalla de una app conocida: banca, streaming, o el portal académico)

Respuestas típicas — todas contradictorias entre sí, y todas defendibles:

- "Se ve elegante" / "es fea"
- "Es rápida", "tiene muchas funciones"
- "Es difícil de usar"
- "Nunca se cae", "cumple lo que promete"
- "Es buena para lo que cuesta"

Nadie está equivocado. Están usando definiciones distintas de calidad sin saberlo.

CONCEPTO · ORIGEN

David Garvin, Harvard, 1984

En plena crisis de calidad de la industria estadounidense frente a la japonesa —cuando "calidad" era el tema de moda en gestión y nadie se entendía— Garvin hizo un movimiento distinto: no propuso una sexta definición mejor.

Argumentó que "calidad" no es un concepto ambiguo: son cinco conceptos distintos que comparten una palabra.

Cada uno lo desarrolló una disciplina distinta, en aislamiento, y ninguna estaba equivocada dentro de su propio dominio:

- **Filosofía** → la definición trascendente
- **Economía** → basada en el producto y basada en el valor
- **Marketing** → basada en el usuario
- **Gestión de operaciones** → basada en la manufactura

"¿Cuál es la verdadera definición de calidad?" es una pregunta mal planteada. Garvin no la resuelve — la disuelve.

CONCEPTO

Garvin: cinco maneras de decir calidad

Enfoque	Calidad es...	Lo dirías así
Trascendente	Excelencia reconocible pero no definible	"Se ve elegante" / "es fea"
Basado en el producto	Mayor cantidad de un atributo deseable	"Es más rápida", "tiene más funciones"
Basado en el usuario	Adecuación a lo que el usuario necesita	"Es difícil de usar"
Basado en la fabricación	Conformidad con la especificación	"Nunca se cae", "cumple lo que promete"
Basado en el valor	Rendimiento aceptable a un costo aceptable	"Es buena para lo que cuesta"

Son las mismas respuestas contradictorias de la diapositiva anterior. Cada una es válida — bajo un enfoque distinto, y sin que quien la dijo supiera cuál estaba usando.

CONCEPTO · ORIGEN

Philip Crosby y "cero defectos"

Crosby no era académico: era ingeniero de control de calidad. En los años 60, en el programa de misiles Pershing de Martin Company, lideró un experimento que cambiaría todo: exigir cero defectos no como aspiración, sino como objetivo de proceso — prevenir en vez de inspeccionar después de que el error ya ocurrió.

Después pasó 14 años como vicepresidente de calidad en ITT, una multinacional con decenas de divisiones en todo el mundo. Ahí aprendió algo que marcó su carrera: a los gerentes no los convencía la estadística, los convencía el dinero.

En 1979 publicó "Quality Is Free" ("La calidad es gratis"), y con eso volvió la calidad un tema de la alta dirección — no solo del área de ingeniería.

"Cero defectos" nació en una línea de producción de misiles, muy controlada. Al llevarlo a software y a gestión general se volvió más un principio motivacional que un método estadístico: a diferencia de Deming, Crosby fija el objetivo pero no entrega las herramientas para alcanzarlo.

Sus ideas se condensan en cuatro afirmaciones — la que sigue es la primera.

CONCEPTO

Crosby: conformidad con los requisitos

Philip Crosby

- Calidad = **conformidad con los requisitos**. No "bondad" ni "elegancia".
- Meta: **cero defectos**
- *"La calidad es gratis"* — lo caro es la no-conformidad
- Ventaja: es medible. Se cuenta lo que no conforma.



CONCEPTO · ORIGEN

Joseph Juran y la mirada del usuario

Juran nació en Rumania en 1904, emigró de niño a Estados Unidos, y empezó su carrera en el departamento de inspección de Western Electric — la misma planta de Hawthorne donde se documentó por primera vez el "efecto Hawthorne". Ahí aplicó el principio de Pareto a los defectos: pocas causas explican la mayoría de los problemas.

En 1954 viajó a Japón, invitado a enseñarle gestión de calidad a la alta dirección — no a los técnicos. Junto con Deming, es una de las dos personas a las que la industria japonesa señala como origen de su revolución de calidad de posguerra. Treinta años después, esa misma revolución fue la crisis que llevó a Garvin a escribir sus cinco definiciones.

Su aporte central fue correr el criterio de calidad del documento al usuario: no importa qué tan bien esté escrita la especificación si no sirve para lo que la gente necesita.

Su otro legado es la "trilogía de Juran" — planificar, controlar, mejorar — tomada del lenguaje del presupuesto financiero, no de la ingeniería. Es una forma de gestión, no un método estadístico: por eso "adecuación al uso" es más difícil de medir que "conformidad con los requisitos", y la industria del testing históricamente se apoyó más en Crosby.

Esa dificultad para medirlo es justo lo que provoca la tensión con Crosby que viene más adelante.

CONCEPTO

Juran: adecuación al uso

Joseph Juran

- Calidad = **adecuación al uso** (*fitness for use*)
- Lo decide quien usa el producto, no quien lo especifica
- Trilogía: planificar · controlar · mejorar
- Ventaja: no se puede "cumplir la especificación" y fracasar al mismo tiempo



TENSIÓN CONCEPTUAL

Dónde chocan

Un sistema de matrícula cumple el 100 % de sus requisitos.

Ningún estudiante logra matricularse sin ayuda.

- Según Crosby: **es de calidad**. Conformar con todo lo especificado.
- Según Juran: **no lo es**. Nadie puede usarlo.

¿Quién tiene razón?



CONCEPTO

Tres lugares donde vive la calidad

Calidad de...	Se pregunta	Se mide con
Producto	¿Cómo es el software construido?	ISO/IEC 25010
Proceso	¿Cómo se construyó?	ISO/IEC 12207, CMMI
Uso	¿Le sirve a quien lo usa, en su contexto?	ISO/IEC 25010 (calidad en uso)

Premisa de todo el curso: la calidad del proceso condiciona la del producto.



CONCEPTO · ESTÁNDARES

ISO/IEC 25010: calidad de producto

Define **8 características** para evaluar cómo está construido el software — la norma que se usará en el taller de hoy para clasificar cada hallazgo.

- **Adecuación funcional:** hace lo que el usuario necesita, de forma completa y correcta.
- **Eficiencia de desempeño:** tiempos de respuesta y uso de recursos adecuados.
- **Compatibilidad:** coexiste con otros sistemas e intercambia información con ellos.
- **Usabilidad:** fácil de entender, aprender, operar y atractiva para el usuario.
- **Confiabilidad:** se mantiene funcionando en condiciones normales y ante fallos.
- **Seguridad:** protege información y datos de accesos no autorizados.
- **Mantenibilidad:** fácil de modificar, corregir y mejorar sin introducir defectos.
- **Portabilidad:** se puede transferir de un entorno de hardware/software a otro.

CONCEPTO · ESTÁNDARES

ISO/IEC 25010: calidad en uso

La misma norma define un segundo modelo, con **5 características**, para cuando el software ya está en manos de alguien resolviendo una tarea real:

- **Efectividad:** el usuario logra sus objetivos con precisión y completitud.
- **Eficiencia:** logra esos objetivos usando pocos recursos (tiempo, esfuerzo).
- **Satisfacción:** el resultado es útil, cómodo de usar y genera confianza.
- **Libertad de riesgo:** no genera consecuencias negativas (económicas, de salud, ambientales).
- **Cobertura de contexto:** funciona bien tanto en el contexto previsto como en otros no previstos.

Diferencia clave: "calidad de producto" se mide sobre el software en sí; "calidad en uso" se mide sobre el resultado de usarlo.



CONCEPTO · ESTÁNDARES

ISO/IEC 12207: procesos del ciclo de vida

No evalúa el software, sino **qué procesos** debería tener una organización para construirlo — desde el acuerdo con el cliente hasta el retiro del sistema.

- **Procesos de acuerdo:** adquisición y suministro entre las partes.
- **Procesos de habilitación organizacional:** gestión del portafolio de proyectos, recursos humanos, infraestructura.
- **Procesos de proyecto:** planeación, control, gestión de riesgos, de la configuración.
- **Procesos técnicos:** requisitos, arquitectura, diseño, implementación, integración, verificación, validación, operación, mantenimiento.

Es un **marco de referencia**: dice qué procesos deberían existir, no qué tan bien se ejecutan — eso lo mide CMMI.

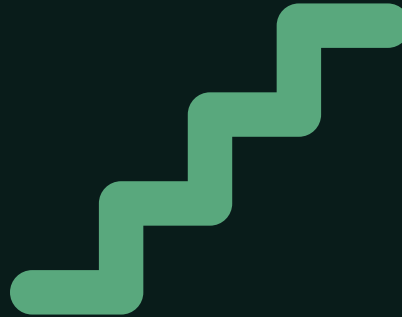
CONCEPTO · ESTÁNDARES

CMMI: qué tan maduro es el proceso

CMMI (Capability Maturity Model Integration) evalúa **qué tan consistente y controlada** es la forma en que una organización ejecuta sus procesos, en 5 niveles:

1. **Inicial:** el trabajo es ad hoc, impredecible; depende de héroes individuales.
2. **Gestionado:** hay planeación y seguimiento básico a nivel de proyecto.
3. **Definido:** los procesos están estandarizados en toda la organización.
4. **Gestionado cuantitativamente:** el desempeño se mide y controla con datos estadísticos.
5. **En optimización:** mejora continua basada en esos mismos datos.

ISO/IEC 12207 dice qué procesos deben existir; CMMI dice qué tan bien y de forma cuán consistente se ejecutan.



CONCEPTO

Qué cuesta la calidad

Precio de la conformidad — lo que se invierte para que salga bien

- Prevención: capacitación, estándares, revisiones de diseño
- Evaluación: pruebas, inspecciones, auditorías

Precio de la no-conformidad — lo que cuesta que salga mal

- Fallas internas: retrabajo antes de entregar
- Fallas externas: soporte, parches, multas, reputación, litigios



CONCEPTO EN DISPUTA

La curva del costo de corregir

Afirmación clásica: corregir un defecto cuesta más cuanto más tarde se encuentra.

Requisitos → Diseño → Código → Pruebas → Producción

(costo creciente por fase)

Advertencia: los multiplicadores concretos que circulan (1 : 10 : 100 : 1000) **no tienen respaldo empírico sólido.**



CASO · ANTECEDENTE

El precedente ignorado

Diez meses antes, en una prueba de recuperación de desastres, un equipo siguió usando datos de prueba el lunes siguiente, ya en producción. Pérdida: **~7,5 millones de dólares**.

La respuesta de Knight fue local: limitar el sistema al horario de mercado, detener cotizaciones tras la primera ejecución, y agregar un ítem a una lista de verificación.

"Knight reaccionó de forma estricta al síntoma, limitando su respuesta a evitar que ese problema puntual se repitiera." — Orden de la SEC

Nadie preguntó si los controles alcanzaban para impedir órdenes erróneas en general. Es el ejemplo de un análisis causal que se detiene en el síntoma.



CIERRE DE CASO

Volver al principio

¿En qué momento se pudo detener Knight Capital?

- Al eliminar el código muerto en 2003–2012
- Al prohibir la reutilización de banderas asociadas a código viejo
- Al exigir revisión de un segundo técnico
- Al automatizar la verificación de que los 8 servidores quedaran iguales
- Al probar el despliegue en un entorno equivalente a producción

212 órdenes de entrada → 4 millones de ejecuciones de salida. Un factor de amplificación de ~18 900x.



CASO · PERSPECTIVA

¿Bastaba con un doble chequeo?

La propia SEC razona en modo contrafactual: un procedimiento simple de doble verificación del despliegue podría haber detectado el servidor faltante y evitado todo lo demás.

Existe una crítica seria a ese razonamiento, desde la ingeniería de resiliencia: los contrafactuales describen un mundo que no existió. Después del accidente siempre se identifica la barrera que faltaba; antes, competía con cientos de barreras igual de plausibles.

La pregunta productiva no es "¿qué barrera faltaba?" sino "¿qué hacía que su ausencia pareciera razonable a gente competente?"



SÍNTESIS

Lo que hay que llevarse de hoy

1. **Error, defecto y falla** son tres cosas distintas y no intercambiables.
2. Un defecto puede existir **sin producir jamás** una falla.
3. **Calidad no tiene una sola definición.** Crosby mide contra la especificación; Juran, contra el uso.
4. Cuando la especificación está mal, **conformar con ella produce un producto defectuoso.**
5. La calidad se juega en el **producto**, en el **proceso** y en el **uso**.



INSTRUCCIONES

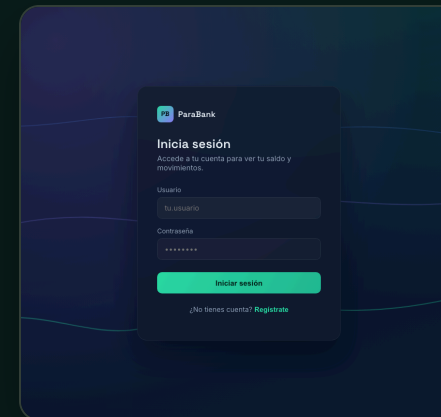
Taller de hoy

Cazar defectos — 2 horas, en sala de Zoom asignada

Cada equipo debe ingresar a la aplicación que se indica al lado. La deben revisar y entregar como mínimo **3 hallazgos**, cada uno con:

- Descripción y pasos de reproducción
- Clasificación: error, defecto o falla — **con justificación**
- Característica ISO/IEC 25010 comprometida
- Generar un archivo markdown con el reporte de errores

Aplicación a probar: parabank-ui.onrender.com · **La entrega cierra al terminar las dos horas.**



CIERRE

Para la próxima semana

Antes de la clase de la semana 2, hay que llegar con esto leído:

- Verificación vs. validación — dos páginas, el modelo en V
- El ciclo de aseguramiento de la calidad

En la semana 2 arrancan las defensas. Dos o tres grupos elegidos al azar explican el trabajo de hoy. El docente escoge quién responde.

